Universidad Anáhuac Mayab

PROYECTO 3 - NAVEGADOR WEB

Estructura de Datos y Algoritmos

Clave de la asignatura: 202660_50560

Profesora: Mtra. Lizbeth Hernández Olan

Amir Moisés Hernández Farah

Emiliano Peniche Correa

Marco Gómez Fuentes

Rodrigo Quijano López

Lenguaje de programación: Python 3 (aplicación de consola)

Repositorio:

https://github.com/Amirhf409/Anahuac-Mayab-Portafolio/tree/main/Estructuras%20de%20Datos/Proyectos/Proyecto3/Codigo

Mérida, Yucatán 20 de septiembre de 2026

Fase 1. Contextualización

¿Qué se Debe Hacer?

Tenemos que programar una aplicación que imite el funcionamiento básico de un navegador web. El historial de navegación se tiene que manejar con la estructura de datos Pila, que trabaja bajo la regla LIFO: el último elemento que entra es el primero que sale. Desde un menú el usuario escribe una URL para visitar una página, regresa con la opción Back, avanza con la opción Forward y cierra el programa con Exit.

Para que el historial funcione en los dos sentidos se necesitan dos pilas trabajando al mismo tiempo:

- Pila atrás (back stack): guarda las páginas por las que el usuario ya pasó antes de la página actual.
- Pila adelante (forward stack): guarda las páginas a las que el usuario puede volver después de haber usado Back.

Las reglas que pide el proyecto son tres. Al visitar una página nueva, la página actual se apila en la pila de atrás, la nueva pasa a ser la actual y la pila de adelante se vacía. Al usar Back, la actual se apila en la pila de adelante y el tope de la pila de atrás se convierte en la actual. Al usar Forward pasa lo contrario.

¿Qué Necesitamos?

- Una clase Pila hecha por nosotros, con un arreglo y un índice llamado tope. El tope vale −1 cuando la pila está vacía y capacidad − 1 cuando está llena, igual al modelo que se vio en clase. Los métodos se nombraron en el mismo estilo que la clase Stack de Java que revisamos (estaVacia, estaLlena, apilar, desapilar, verTope).
- Una clase que guarde la información de una página: su dirección y el texto que se muestra en pantalla.
- Una clase Navegador que controle las dos pilas y la página actual, y que tenga las reglas de visitar, retroceder y avanzar.
- Un menú de consola que muestre en todo momento la página actual y el contenido de las dos pilas.

- Python 3.

Elementos

Elemento identificado	Cómo se representa en el sistema
Página web	Objeto de la clase PaginaWeb, con su URL y su descripción.
Página que no existe	Objeto de la clase PaginaNoEncontrada (error 404), que hereda de Pagina.
Historial de retroceso	Objeto de la clase Pila llamado pilaAtras.
Historial de avance	Objeto de la clase Pila llamado pilaAdelante.
Página actual	Atributo paginaActual de la clase Navegador.
Operaciones del usuario	Métodos visitar(), retroceder() y avanzar() de la clase Navegador.
Menú del programa	Funciones del archivo main.py.
Catálogo de sitios	Diccionario CATALOGO en navegador.py. Se muestra en pantalla antes de pedir la URL.

Fase 2. Definición de Requerimientos Funcionales

Del contexto planteado se obtuvieron ocho requerimientos funcionales. Los primeros cinco vienen directo de las opciones del menú y los últimos tres del apartado de validaciones y mensajes.

Clave	Requerimiento
R1	Visitar una página nueva
R2	Regresar a la página anterior (Back)
R3	Avanzar a la página siguiente (Forward)
R4	Mostrar la página actual en todo momento
R5	Salir del programa (Exit)
R6	Evitar errores cuando no hay páginas disponibles

Clave	Requerimiento
R7	Limpiar la pila de avance al visitar una página nueva
R8	Mostrar mensajes claros y amigables

Aquí se documenta cada requerimiento con el esquema de nombre, descripción, entrada y salidas.

Nombre	R1 – Visitar una página nueva
Descripción	El usuario elige la opción 1 del menú. El programa le muestra la lista de sitios que conoce el navegador y le pide escribir una URL. La página actual se apila en la pila de atrás, la página nueva pasa a ser la página actual y la pila de adelante se vacía.
Entrada	Opción 1 y una URL, por ejemplo https://www.google.com
Salidas	La nueva página se muestra como página actual y la pila de atrás crece en un elemento.

Nombre	R2 – Regresar a la página anterior (Back)
Descripción	El usuario elige la opción 2. La página actual se apila en la pila de adelante y el tope de la pila de atrás se convierte en la página actual.
Entrada	Opción 2 del menú.
Salidas	Se muestra la página anterior y las dos pilas se actualizan en pantalla.

Nombre	R3 – Avanzar a la página siguiente (Forward)
Descripción	El usuario elige la opción 3. La página actual se apila en la pila de atrás y el tope de la pila de adelante se convierte en la página actual.
Entrada	Opción 3 del menú.
Salidas	Se muestra la página siguiente y las dos pilas se actualizan en pantalla.

Nombre	R4 – Mostrar la página actual en todo momento
Descripción	Antes de cada menú el programa imprime la página actual con su descripción y el contenido completo de las dos pilas, señalando cuál elemento es el tope y cuántos espacios están ocupados.
Entrada	Cualquier operación del navegador.
Salidas	Pantalla con la página actual, la pila de atrás y la pila de adelante.

Nombre	R5 – Salir del programa (Exit)
Descripción	La opción 4 termina el ciclo del menú y cierra el programa con un mensaje de despedida.
Entrada	Opción 4 del menú.

Salidas	El programa termina su ejecución.

Nombre	R6 – Evitar errores cuando no hay páginas disponibles
Descripción	Antes de retroceder o avanzar se revisa con estaVacia() si la pila correspondiente tiene elementos, y antes de apilar se revisa con estaLlena() que todavía haya espacio. Si la operación no se puede hacer, el programa avisa y sigue funcionando en lugar de detenerse.
Entrada	Opción 2 o 3 sin páginas disponibles, u opción 1 con la pila de atrás llena.
Salidas	Mensaje PILA VACIA o PILA LLENA, sin que el programa se cierre.

Nombre	R7 – Limpiar la pila de avance al visitar una página nueva
Descripción	Cada vez que se visita una página nueva se llama al método vaciar() de la pila de adelante, porque desde esa página ya no existe un camino hacia adelante.
Entrada	Opción 1 con una URL válida.
Salidas	La pila de adelante queda vacía.

Nombre	R8 – Mostrar mensajes claros y amigables
Descripción	Después de cada operación el programa imprime un mensaje que empieza con >> explicando lo que pasó. Si la URL no está en el catálogo, la página se muestra como error 404. Si el usuario no escribe nada o teclea una opción que no existe, se le indica qué se esperaba.
Entrada	Cualquier operación del navegador.
Salidas	Mensaje de resultado o de error en la consola.

Fase 3. Diseño

El sistema se dividió en cinco clases repartidas en cuatro archivos. La clase Pila es la estructura de datos; Pagina, PaginaWeb y PaginaNoEncontrada representan lo que se muestra en pantalla, y Navegador es la que junta todo y aplica las reglas del historial.

Se usó herencia porque el navegador tiene que guardar dos tipos de página en la misma pila: las que sí existen en el catálogo y las que no. Como las dos heredan de Pagina, la pila las trata igual y cada clase hija responde con su propia descripción cuando se le pide.

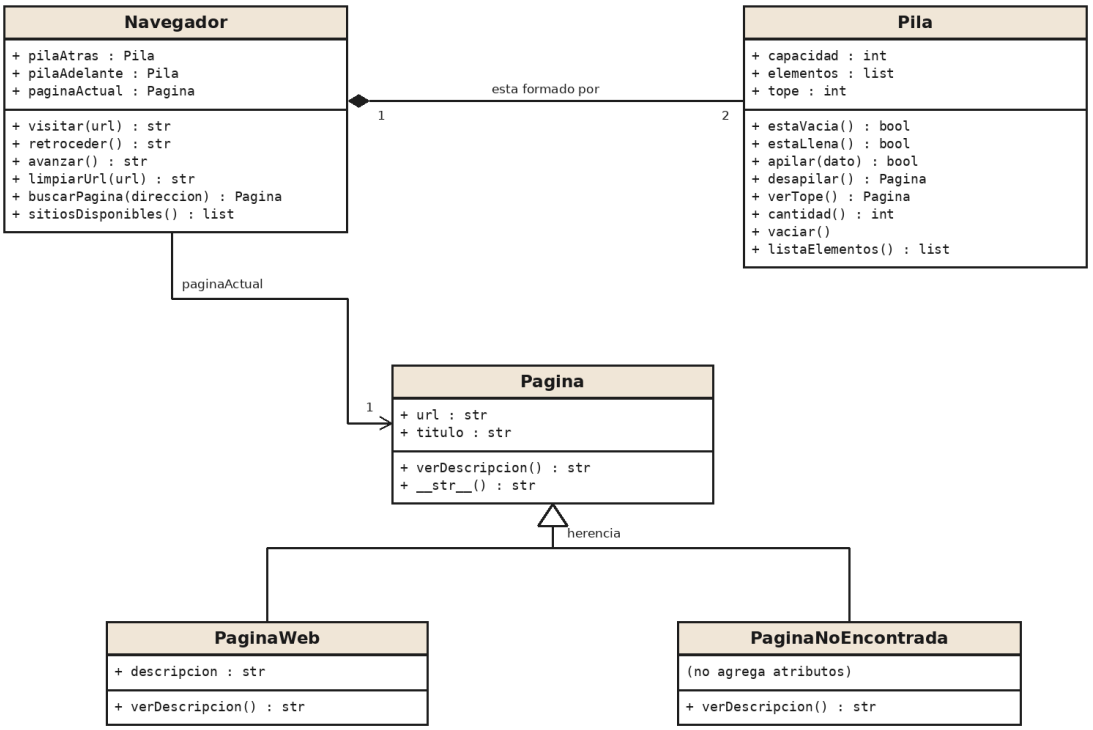


Figura 1. Diagrama de clases del proyecto. El triángulo blanco marca la herencia y el rombo relleno la composición.

Aquí utilicé o representé el triángulo blanco como la herencia: PaginaWeb y PaginaNoEncontrada sobrescriben verDescripcion(). El rombo relleno indica que el Navegador crea y contiene sus dos objetos de la clase Pila.

Clases, Atributos y Métodos

Las cinco clases con sus atributos y sus métodos quedaron así:

Pila
+ capacidad : int + elementos : list + tope : int
+ estaVacia() : bool + estaLlena() : bool + apilar(dato) : bool + desapilar() : Pagina + verTope() : Pagina + cantidad() : int + vaciar() + listaElementos() : list

Pagina
+ url : str + titulo : str
+ verDescripcion() : str + __str__() : str

PaginaWeb (hereda de Pagina)
+ descripcion : str
+ verDescripcion() : str

PaginaNoEncontrada (hereda de Pagina)
(no agrega atributos nuevos)
+ verDescripcion() : str

Navegador
+ pilaAtras : Pila + pilaAdelante : Pila + paginaActual : Pagina
+ visitar(url) : str + retroceder() : str + avanzar() : str + limpiarUrl(url) : str + buscarPagina(direccion) : Pagina + sitiosDisponibles() : list

Relaciones Entre las Clases

- Herencia: PaginaWeb y PaginaNoEncontrada heredan de Pagina y sobrescriben el método verDescripcion(). El método buscarPagina() del navegador devuelve una o la otra según el caso, pero las dos son de tipo Pagina.
- Composición: la clase Navegador está formada por dos objetos de la clase Pila, que se crean en su constructor y no existen fuera de él.
- Asociación: el atributo paginaActual del Navegador apunta a un objeto de tipo Pagina, que va cambiando conforme el usuario navega.

Fase 4. Codificación

El programa se escribió en Python 3 y se separó en cuatro archivos para que cada clase quedara en su lugar:

Archivo	Contenido
pila.py	Clase Pila. Implementación propia de la estructura LIFO con arreglo y tope.
pagina.py	Clases Pagina, PaginaWeb y PaginaNoEncontrada. Aquí está la herencia.
navegador.py	Clase Navegador: las dos pilas, la página actual y el catálogo de sitios.
main.py	Menú de consola y las funciones que imprimen la página actual y las pilas.

Implementación de los Métodos

La pila se programó con un arreglo y un índice llamado tope, tal como lo vimos en clase, en el ejemplo particularmente de Java. Apilar y desapilar revisan primero si la pila está llena o vacía, así nunca se produce un desbordamiento ni un subdesbordamiento:

```
def apilar(self, dato):
    if self.estaLlena():
        return False
    self.tope = self.tope + 1
    self.elementos[self.tope] = dato
    return True

def desapilar(self):
    if self.estaVacia():
        return None
```

```
    dato = self.elementos[self.tope]
    self.elementos[self.tope] = None
    self.tope = self.tope - 1
    return dato
```

El método visitar() es el que aplica las tres reglas del proyecto: apila la página actual, cambia la página actual y vacía la pila de adelante.

```
if self.paginaActual != None:
    if self.pilaAtras.estaLlena():
        return "PILA LLENA: ..."
    self.pilaAtras.apilar(self.paginaActual)      # push

self.paginaActual = self.buscarPagina(direccion)
self.pilaAdelante.vaciar()                    # se limpia el avance
```

Los métodos retroceder() y avanzar() son simétricos: uno apila en la pila de adelante y desapila de la de atrás, y el otro hace exactamente lo contrario.

```
def retroceder(self):
    if self.pilaAtras.estaVacia():
        return "PILA VACIA: no hay paginas anteriores para regresar."
    if self.pilaAdelante.estaLlena():
        return "PILA LLENA: el historial de adelante ya esta lleno."
    self.pilaAdelante.apilar(self.paginaActual)      # push
    self.paginaActual = self.pilaAtras.desapilar()   # pop
    return "Regresaste a " + self.paginaActual.url
```

Asignaciones y Condicionales

- Asignaciones: el tope sube o baja de uno en uno con cada push y cada pop; paginaActual recibe el objeto que devuelve buscarPagina() o desapilar(); al desapilar, la casilla del arreglo se pone en None para no dejar la referencia vieja.
- Condicionales: revisan si la pila está vacía o llena, si el usuario no escribió nada, si la URL aparece en el catálogo y qué opción del menú se eligió.
- Herencia: buscarPagina() devuelve un objeto PaginaWeb o uno PaginaNoEncontrada según el caso, y el resto del programa los usa igual porque los dos son de tipo Pagina.

Documentación del código

Cada clase lleva su comentario y los métodos donde pasa algo que no se entiende tan facilmente, y dentro de la lógica solo añadí comentarios en las partes donde se pone más complicado entender sin ellos: las validaciones de pila vacía y pila llena, el vaciado de la pila de avance, la limpieza de la URL y el recorrido invertido con el que se imprimen las pilas.

Repositorio

El código fuente completo está publicado en GitHub, dentro del portafolio de la materia:

https://github.com/Amirhf409/Anahuac-Mayab-Portafolio/tree/main/Estructuras%20de%20Datos/Proyectos/Proyecto3/Codigo

Fase 5. Pruebas

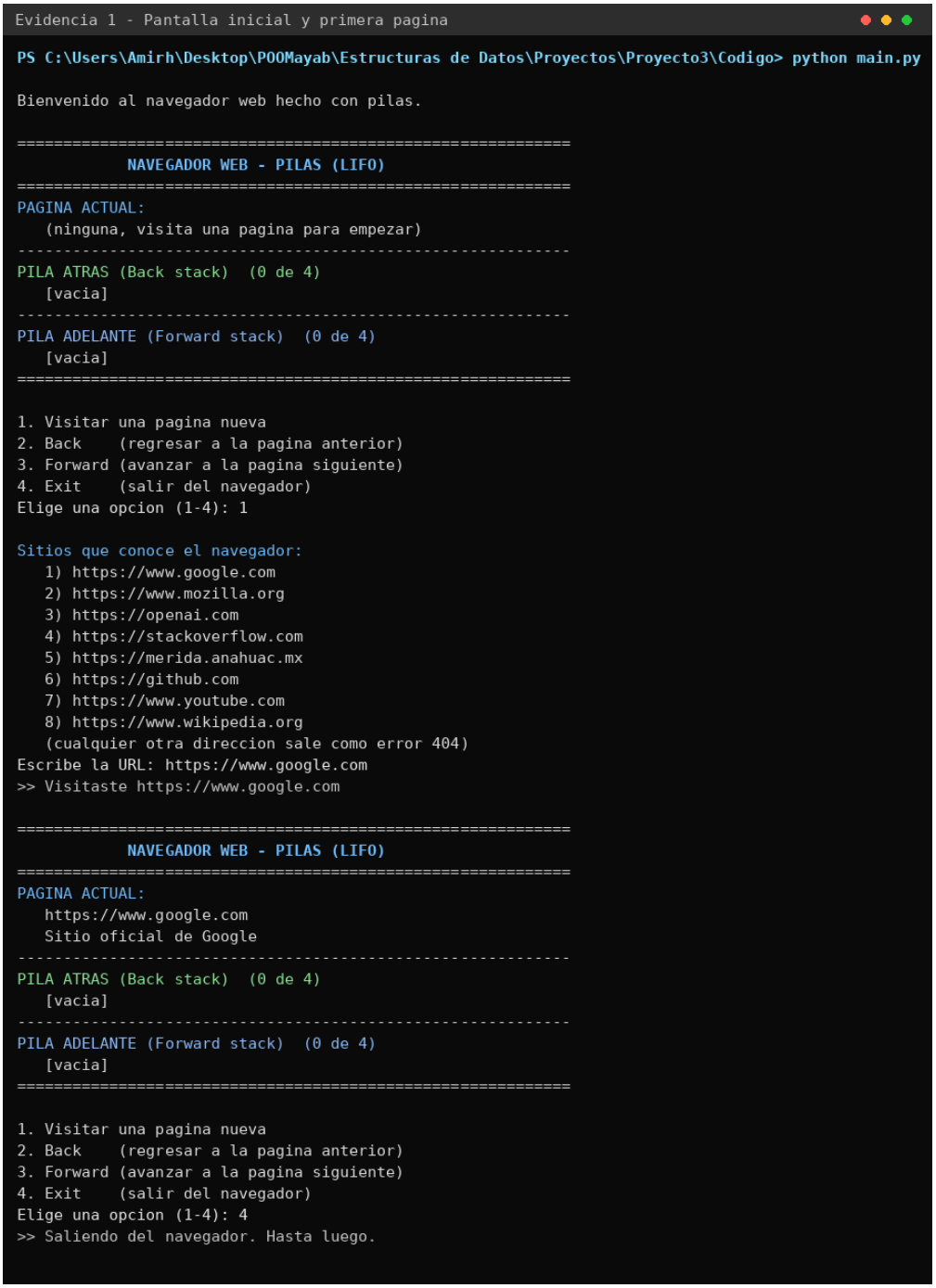
Verificación de los Requerimientos

Se probó cada uno de los requerimientos de la Fase 2, incluyendo los casos en los que el usuario se equivoca. La siguiente tabla concentra los casos de prueba, contra qué requerimiento van y el resultado que se obtuvo:

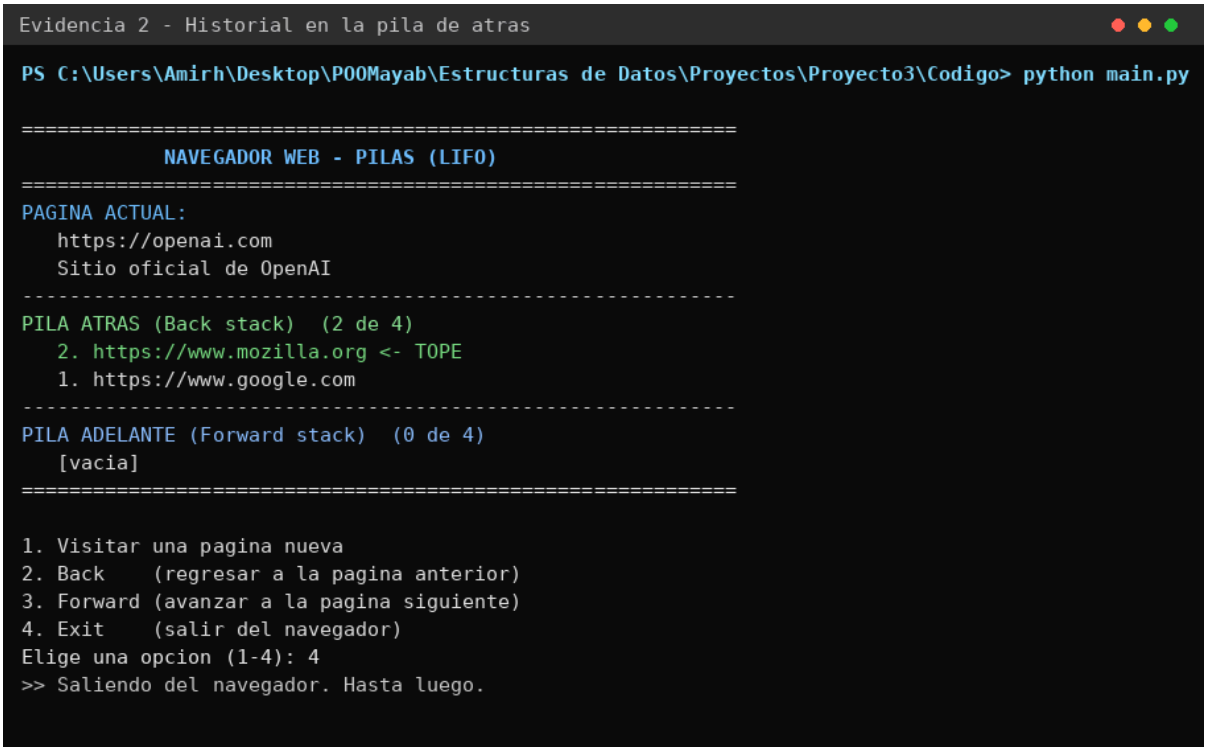
Caso	Prueba realizada	Resultado esperado	Requerimiento	Resultado
P1	Visitar una página con el navegador recién abierto	Se muestra como página actual y la pila de atrás sigue vacía	R1, R4	Correcto
P2	Visitar cuatro páginas seguidas	Las anteriores quedan en la pila de atrás en orden LIFO	R1	Correcto
P3	Opción 2 (Back) con historial disponible	Se regresa a la anterior y la actual pasa a la pila de adelante	R2	Correcto
P4	Opción 3 (Forward) después de un Back	Se avanza otra vez y la actual regresa a la pila de atrás	R3	Correcto
P5	Opción 2 con la pila de atrás vacía	Mensaje PILA VACIA y el programa no se cierra	R6	Correcto
P6	Opción 3 con la pila de adelante vacía	Mensaje PILA VACIA y el programa no se cierra	R6	Correcto
P7	Visitar una página nueva después de retroceder	La pila de adelante se vacía	R7	Correcto
P8	Visitar una URL que no está en el catálogo	Se muestra la página de error 404	R8	Correcto
P9	Visitar con la pila de atrás llena (4 elementos)	Mensaje PILA LLENA y no se cambia de página	R6	Correcto
P10	Opción 1 sin escribir ninguna URL	Mensaje pidiendo escribir una URL	R8	Correcto
P11	Escribir una opción fuera del rango 1-4	Mensaje de opción no válida y el menú se repite	R8	Correcto
P12	Opción 4 (Exit)	El programa se despide y termina	R5	Correcto

Evidencias de la Ejecución

Las siguientes capturas se tomaron corriendo el programa desde la terminal con el comando python main.py.



Evidencia 1. Pantalla inicial: no hay página actual y las dos pilas están vacías. Al elegir la opción 1 el programa muestra los sitios disponibles y se visita la primera página (R1 y R4).



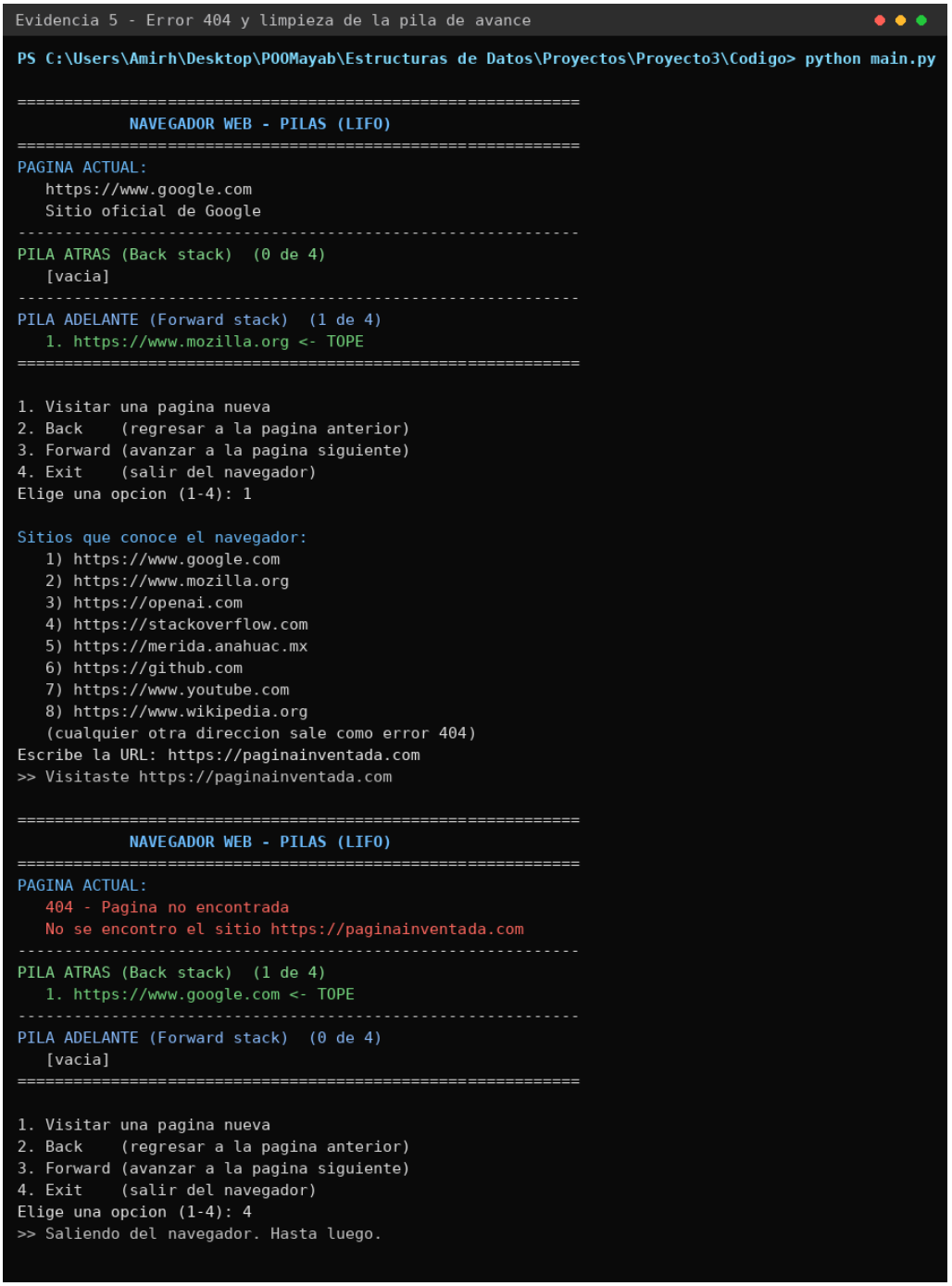
Evidencia 2. Después de visitar tres páginas, la pila de atrás guarda las anteriores y marca cuál elemento es el tope (R1).



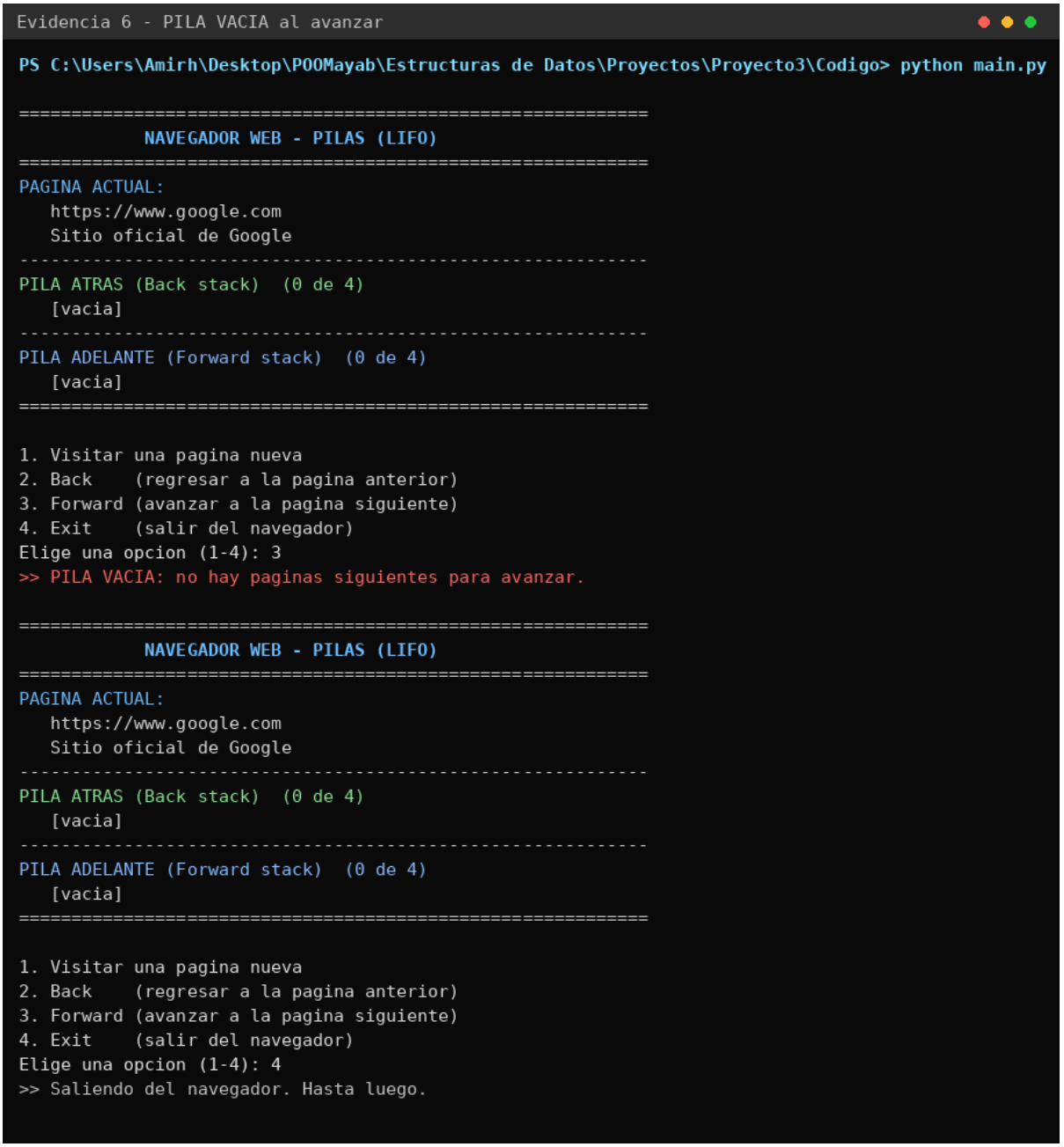
Evidencia 3. Con la opción 2 la página actual pasa a la pila de adelante y el tope de la pila de atrás se vuelve la página actual (R2).



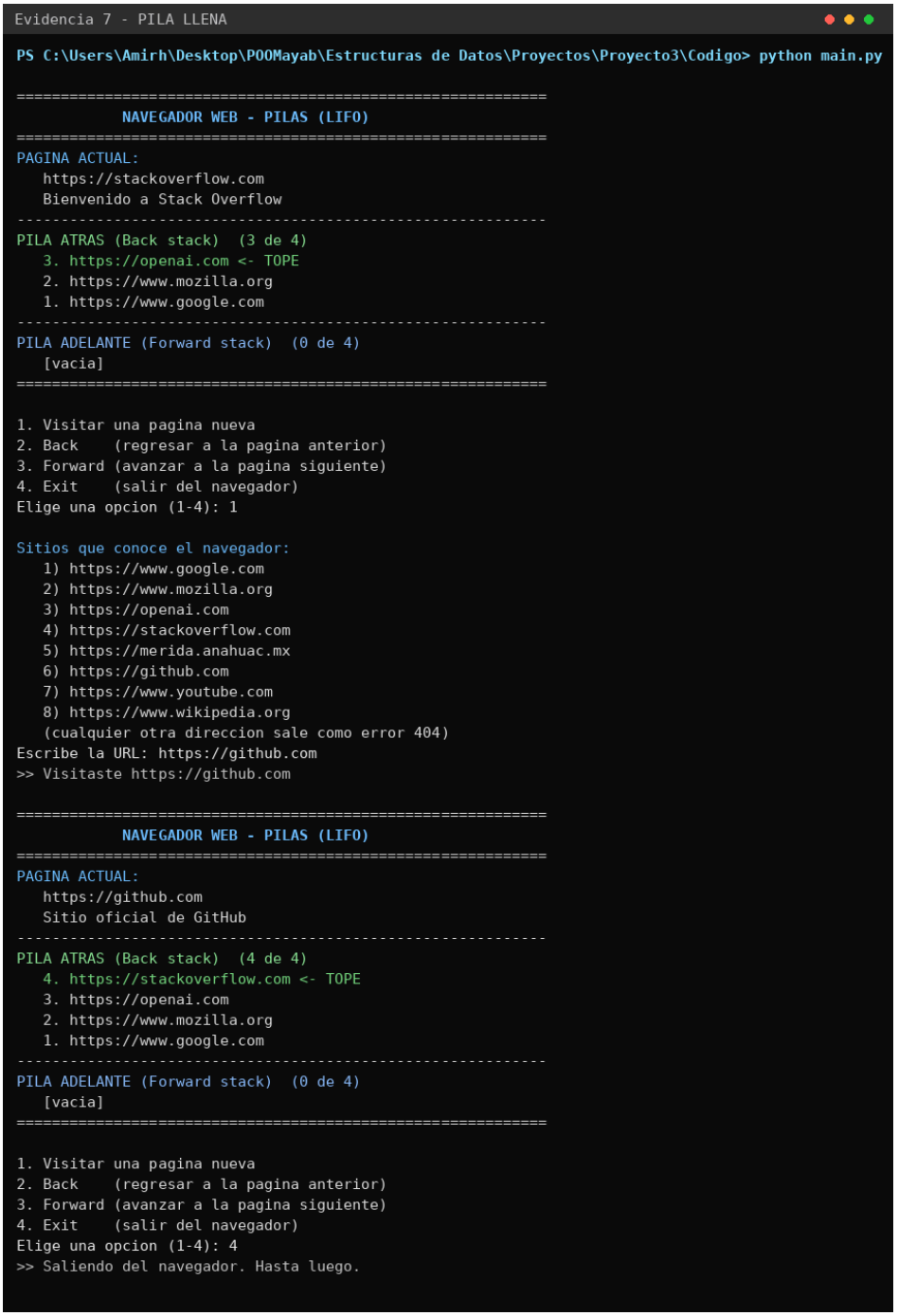
Evidencia 4. Con la opción 3 pasa lo contrario: la actual regresa a la pila de atrás y se avanza (R3).



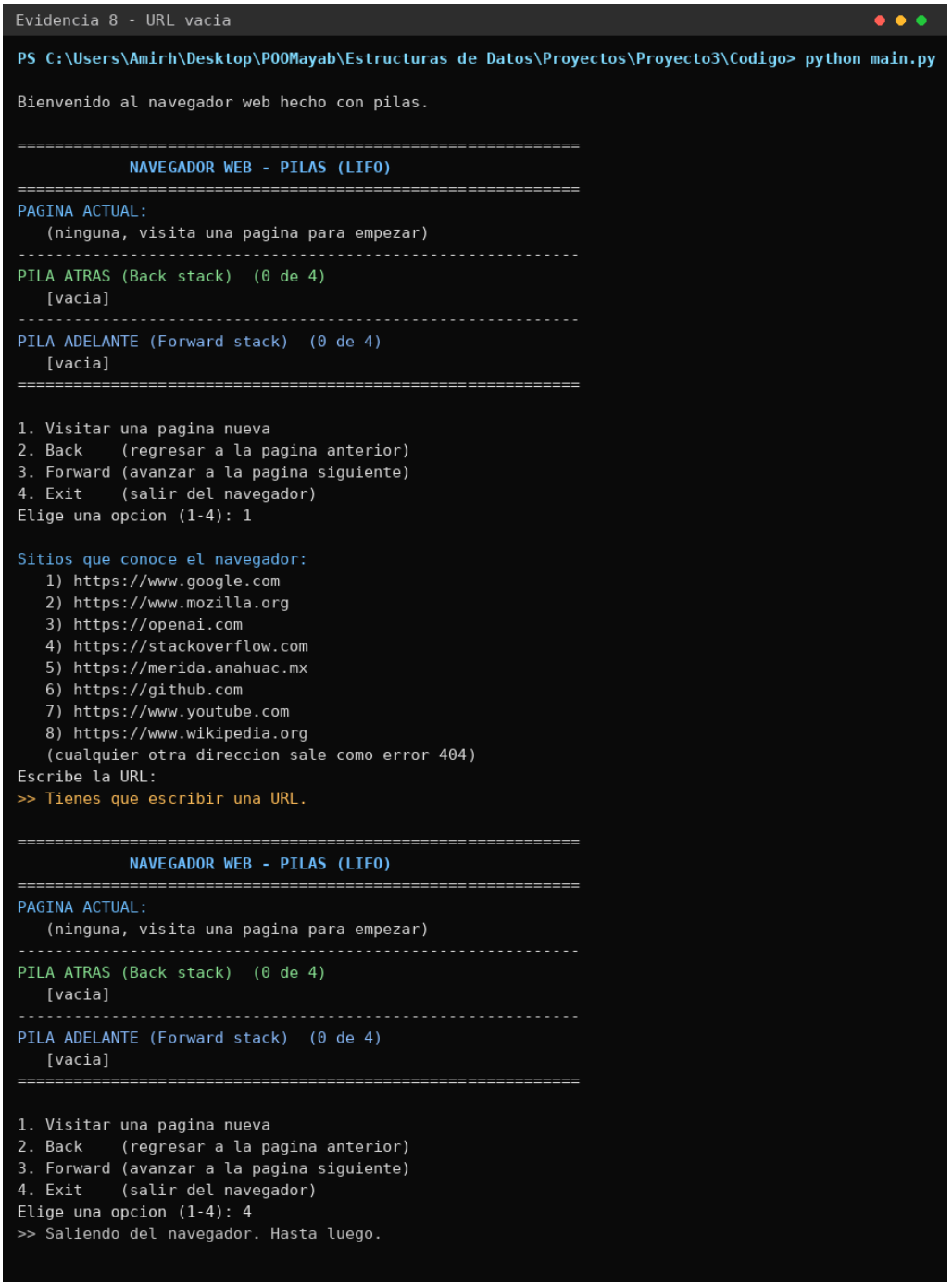
Evidencia 5. Una URL que no está en el catálogo se muestra como error 404, y al tratarse de una página nueva la pila de adelante se vacía (R7 y R8).



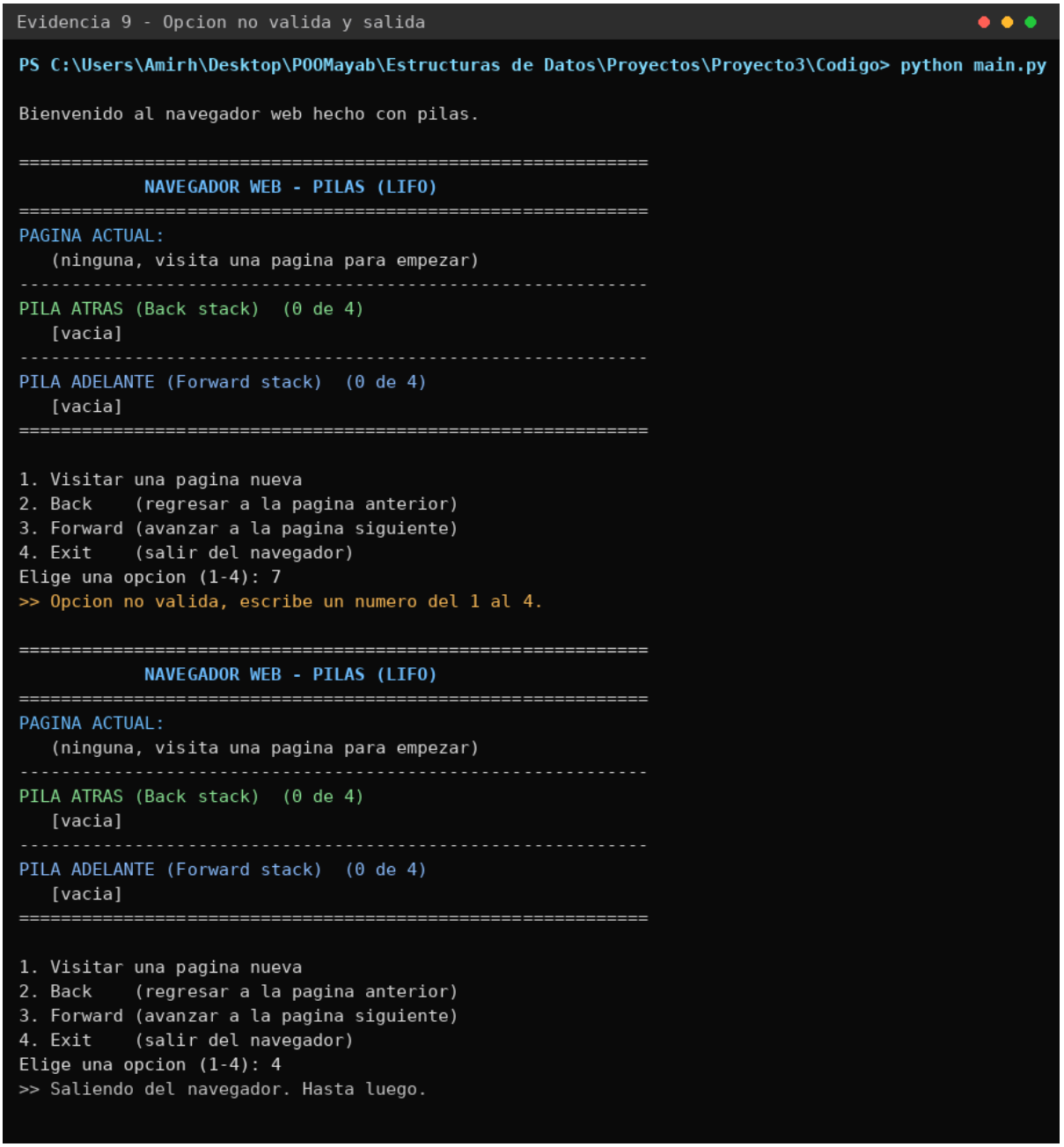
Evidencia 6. Al intentar avanzar sin páginas disponibles aparece el mensaje PILA VACIA y el programa continúa (R6).



Evidencia 7. Con la pila de atrás en su límite de cuatro páginas, el programa avisa con PILA LLENA antes de perder la página actual (R6).



Evidencia 8. Si el usuario no escribe nada, el programa se lo indica en lugar de fallar (R8).



Evidencia 9. Una opción fuera del rango 1-4 se avisa y el menú se repite; la opción 4 cierra el programa (R5 y R8).

Conclusión

El programa cumple con los ocho requerimientos que se definieron en la Fase 2. Usar dos pilas resultó natural para este problema, porque el comportamiento LIFO es justo el que se espera de las opciones Back y Forward: la última página visitada es la primera a la que se regresa.

Lo que más me costó fue entender que la pila de adelante se tiene que vaciar al visitar una página nueva. Al principio no lo hacía y el navegador dejaba avanzar a páginas que ya no tenían nada que ver con el camino actual, igual que si en un navegador de verdad regresara y luego abriera otro sitio. También me sirvió ponerle un límite de cuatro páginas a cada pila, porque así se puede ver en pantalla qué pasa cuando una pila se llena, algo que con una lista de Python nunca hubiera notado.

Las validaciones hechas con estaVacia() y estaLlena() son las que evitan que el programa se caiga. En lugar de dejar que Python marque un error de índice, el navegador avisa con un mensaje y regresa al menú, que es lo que haría cualquier programa terminado.

NOTA:

Hola maestra, en este proyecto utilicé Claude (Anthropic) como herramienta de apoyo, así como en un inicio inteligencia artificial de pycharm (esta última siendo la que menos utilicé). Me ayudé sinceramente en distintos aspectos. Cabe aclarar únicamente cosas de código puesto que el presente documento fue redactado por mi persona. Utilicé inteligencia artificial para escribir código y sugerir auto completaciones de código en el IDE que yo uso, dado que en varias secciones del proyecto me quedaba estancado, pese a recurrir a material de la clase y/o videos de apoyo. La IA me ayudó a construir el código, entender la lógica entre mis clases, profundizar en mis conocimientos acerca de pilas y por último a corregir errores que me habría tomado mucho tiempo descubrir. 🙂

Referencias

Hernández Olan, L. (2026a). Documentación del proyecto [Plantilla de entrega]. Estructuras de datos y algoritmos (202660_50560). Brightspace, Universidad Anáhuac Mayab.

Hernández Olan, L. (2026b). Java – Stack [Material del módulo 4]. Estructuras de datos y algoritmos (202660_50560). Brightspace, Universidad Anáhuac Mayab.

Hernández Olan, L. (2026c). Navegador web: proyecto de pila (LIFO – Last In, First Out) [Especificación del proyecto]. Estructuras de datos y algoritmos (202660_50560). Brightspace, Universidad Anáhuac Mayab.

Hernández Olan, L. (2026d). Pilas [Presentación del módulo 4]. Estructuras de datos y algoritmos (202660_50560). Brightspace, Universidad Anáhuac Mayab.